Fast GPI

1 Motivation

Fast graphics with GPI (so without DIVE).

A port from CPP to fpc Pascal (FPC 3.2.2).

2 C code

2.1 Background

The program shows a dynamic image in grayscales.

Original code is published here: https://github.com/OS2World/DEV-SAMPLES-C-PM-FastGPI

2.2 Array dimensions (bitmap and colors)

To set the colors the following are declared:

- FastGPI: BYTE RGBmap[32];
- FastGPI: BYTE Bitmap[NUM_MASSES_X*NUM_MASSES_Y];
- window_func: LONG palette[33];

With these declarations the size (number of elements) is:

- 32 for RGBmap
- NUM_MASSES_X*NUM_MASSES_Y for Bitmap (size of image in pixels)
- 33 for palette

And typically indices run from 0 to size-1.

Now looking at code:

- in PrepareGraphics: at end:
  - for (x = 0; x < 33; x++) { RGBmap[x] = Bitmap[x]; }
  - so RGBmap must have 33 elements, not 32.
- in DisplayPlane:
  - index for RGBmap runs from 0 to 32.
  - so RGBmap must have 33 elements, not 32.

- in window_func, for palette:
  - elements 0-31 are set in a for loop; element 32 is set separately.

2.3 Arrays (data)

Two files handle arrays for data:

- A-FLOAT.C
- A-FLOAT.H

Arrays are allocated on the heap.

Arrays come in 3 flavours: 1-, 2- or 3-dimensional. The program typically selects a 2D subset of 3D data (node) to deal with by setting pointers. The sizes per dimension are:

- 3
- NUM_MASSES_X
- NUM_MASSES_Y

and so indices run from 0 to size-1:

- 3-1=2
- NUM_MASSES_X-1
- NUM_MASSES_Y-1.

2.4 Color handling

The idea is to present grayscales; to this end the arrays palette and RGBmap are used. Colors are 24 bit colors (RGB) so per pixel (32 bits available) the highest byte is always zero.

Code: window_func:

```
/* Set up gray-scale palette for screen image. */
for (i = 0; i < 32; i++)
{
  j = i << 3;
  palette[i] = (j << 16) | (j << 8) | j;
}
palette[32] = 0x00ffffffL;
```

The palette[i] assignment ensures that R=G=B so a grayscale. Variable j fits in 8 bits (max 31 + shl 3).

Overview of color handling steps:

- window_func: Set up gray-scale palette for screen image.
- PrepareGraphics: Determine mapping from logical color value to bitmap color table index. This sets 33 (!) values of RGBmap.
- DisplayPlane: set Bitmap[y*NUM_MASSES_X+x] = RGBmap[disp_val]; with disp_val limited to the interval [0..32]. The value comes from the wave equation (which starts with 30*sin(…))..

2.5 Compilation

I modified the code to compile with Watcom C++ 1.9, as follows (all in FASTGPI.C):

- insert EXPENTRY in void Model(ULONG); to get void EXPENTRY Model(ULONG); (this removes one error)
- insert return 0; at end (this removes one warning).

Watcom:

- set compiler for multi-threaded application

2.6 Running the program

No problem :). Use the Pause key to “freeze” the image. Colors are grayscales. Alt-F4 ends the program.

3 Pascal code (fpc)

3.1 Variable names

Pascal is not case-sensitive so I inserted an underscore after h. Example: hab becomes h_ab.

3.2 Array dimensions (bitmap and colors)

To set the colors the following were declared:

- FastGPI: BYTE RGBmap[32];
- FastGPI: BYTE Bitmap[NUM_MASSES_X*NUM_MASSES_Y];
- window_func: LONG palette[33];

The C dimensions have been copied and parameterized, except RGBmap: 33 elements, not 32:

- FastGPI: BYTE RGBmap[nrgb];
- window_func: LONG palette[npal];

With consts nrgb=33 and npal=33.

3.3 Arrays (data)

For fpc we have a couple of options:

- use dynamic arrays (whose index is also zero-based; and they are created on the heap)
- use fixed arrays (the index ranges are known)
- use subsets as (more or less) in C or just use indices for this purpose

I went for dynamic arrays and indices for subsets.

3.4 Color handling

Have a var in the PrepareGraphics header!

3.5 Other

Replace | (bitwise or): can be done by or or by +. Note that || stands for logical or.

3.6 Running the program

No problem :). Use the Pause key to “freeze” the image. Alt-F4 does NOT end the program.

Executable seems slower than its C counterpart? Hint: try to run 2 instances...

3.7 Unit issues: pmgpi

To some extent a matter of style.

We have in C:

```
hbm = GpiCreateBitmap(hpsMemory, &bmih, 0L, NULL, NULL);
```

and in Pascal (dummy variables xxx*)

```
h_bm := GpiCreateBitmap(h_psMemory, bmih, 0, xxx3 {0}, xxx2);
```

where I would prefer

```
h_bm := GpiCreateBitmap(h_psMemory, bmih, 0, nil, nil);
```

So need to change GpiCreateBitmap in pmgpi from

```
function GpiCreateBitmap(hps : cardinal;var pbmpNew : Tbitmapinfoheader2;flOptions :
cardinal;var pbInitData : BYTE;var pbmiInfoTable : Tbitmapinfo2) : cardinal; cdecl;
```

to:

```
function GpiCreateBitmap(hps : cardinal;var pbmpNew : Tbitmapinfoheader2;flOptions :
cardinal; pbInitData : pBYTE; pbmiInfoTable : PBitmapinfo2) : cardinal; cdecl;
```

This results in “cleaner” Pascal code without the need to declare dummies (and initialize them).

3.8 Quality

Program can be compiled with -Cr -gh; the result runs.

4 Revisions

[20261106] public version.